

ARBITRAGE BETTING

Python Implementation Using Betting APIs

A complete engineering guide to building an arbitrage betting system in Python. Covers the API landscape (The Odds API for odds discovery, Betfair Exchange API for programmatic bet placement), data normalisation across bookmakers, the arbitrage detection algorithm, stake sizing, alerting, order management, backtesting, and production concerns including rate limits, retries, latency, and logging.

Companion volume: Arbitrage Betting — Theory & Practice

Contents

1. System Overview and Architecture
2. The API Landscape
3. Environment Setup
4. Data Models
5. Fetching Odds — The Odds API
6. Normalising Across Bookmakers
7. The Arbitrage Detection Algorithm
8. The Stake Calculator
9. The Full Scanner Pipeline
10. Alerting via Telegram
11. The Betfair Exchange API
12. Placing Bets Programmatically
13. Order Management and Status Tracking
14. Persistence and Logging
15. Backtesting Framework
16. Rate Limits, Retries, and Robustness
17. Latency and Execution
18. Deployment
19. Legal and ToS Considerations
20. Project Layout and Next Steps

1. System Overview and Architecture

An arbitrage betting system decomposes into six loosely coupled stages. Building each as an independent component with clean interfaces makes the whole system testable, debuggable, and swappable — you can replace the odds source without touching the arb detector, and replace the executor without touching the scanner.

1.1 The pipeline

1. Fetch. Poll one or more odds APIs and pull the latest odds for the sports and markets you follow. Cache aggressively to respect rate limits.

2. Normalise. Convert bookmakers' varied JSON shapes into a canonical internal representation: event → market → outcomes → prices. Reconcile event identity across sources (bookie A's 'Man City' may be bookie B's 'Manchester City FC').

3. Detect. Scan the normalised odds tape for arbs — combinations of outcomes across bookmakers where the combined implied probability is less than 1.

4. Size. For every candidate arb, compute equal-profit stakes given your target total stake and constraints (per-book limits, minimum stakes, rounding).

5. Alert / Execute. Push the opportunity to a human (Telegram, email) for manual placement, or place bets programmatically where the API permits it (realistically: Betfair Exchange for the hedge leg).

6. Track. Log every candidate, every placement, and every settlement. Reconcile against expected P&L. This is where you catch bugs and quantify void rates.

1.2 Directory layout

```
arbengine/
■■■ config.py           # API keys, thresholds, book list
■■■ models.py          # Event, Outcome, Price, Arb dataclasses
■■■ sources/
■   ■■■ odds_api.py    # The Odds API client
■   ■■■ betfair.py     # Betfair Exchange client
■   ■■■ base.py        # Abstract Source interface
■■■ normalise.py       # Event/team name reconciliation
■■■ detector.py        # Arb detection algorithm
■■■ sizing.py          # Stake calculation
■■■ alerts.py          # Telegram/email notifiers
■■■ executor.py        # Betfair order placement
■■■ storage.py         # SQLite persistence
■■■ backtest.py        # Historical replay
■■■ scanner.py         # Orchestrates the pipeline
■■■ main.py            # Entry point
■■■ tests/
■   ■■■ test_detector.py
■■■ .env               # API keys (never commit)
■■■ requirements.txt
■■■ README.md
```

Every file is small; nothing is more than a few hundred lines. Prefer many small focused modules over a few large ones — arbitrage systems churn constantly and you will refactor often.

2. The API Landscape

There is no single API that both surfaces best-of-market odds and lets you place bets everywhere. You will combine a data source (or aggregator) for detection with an execution venue for the actual bets. This is the realistic picture as of 2026:

2.1 Odds data sources

2.2 Execution venues

The overwhelming majority of bookmakers **prohibit programmatic betting** in their terms of service. Automating bet placement at Bet365, William Hill, DraftKings and similar is a ToS violation that will get your account closed. In practice this leaves:

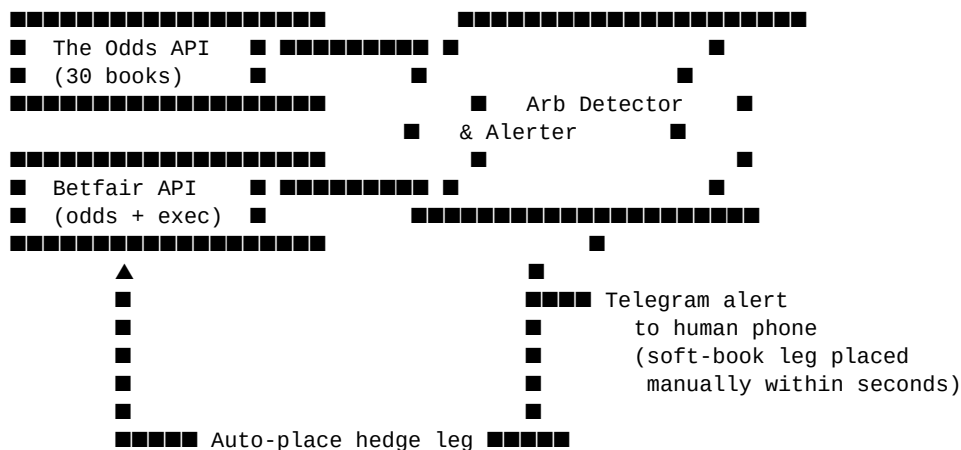
- **Betfair Exchange** — officially supports API betting, well documented, robust.
- **Smarmets** — has an API in beta; more limited but functional for basic use.
- **Matchbook** — API available on request; smaller market.
- **US sports API-first books** — Kalshi, some DFS platforms; different regulatory regime.

For UK-based operations, Betfair Exchange is effectively the only production-grade programmable execution venue. This document uses Betfair for the executable examples in §11–13.

Warning. Do **not** attempt to automate placement at bookmakers whose ToS forbid it. In addition to account closure, you may forfeit any balance held on the platform, and in extreme cases (using scripts to evade fraud controls) you may commit a criminal offence under the Computer Misuse Act 1990. For soft-book arbs, the workflow is: automated detection, Telegram alert, manual placement by a human.

2.3 The pragmatic architecture

Given the above, a realistic solo-arb architecture is:



3. Environment Setup

3.1 Python version and virtualenv

Use Python 3.11 or later. Create an isolated environment:

```
python3.11 -m venv .venv
source .venv/bin/activate    # macOS/Linux
# .venv\Scripts\activate     # Windows
```

```
pip install --upgrade pip
```

3.2 Dependencies

```
# requirements.txt
requests>=2.31          # HTTP client for REST APIs
httpx>=0.27             # async HTTP client (for concurrent fetching)
python-dotenv>=1.0      # load .env
pydantic>=2.7           # data validation
rapidfuzz>=3.9          # fuzzy name matching
betfairlightweight>=2.20 # Betfair API wrapper (Python community)
python-telegram-bot>=21 # Telegram alerts
SQLAlchemy>=2.0         # ORM for storage
apscheduler>=3.10       # scheduled polling
tenacity>=8.3           # retry with backoff
loguru>=0.7             # nicer logging
python-dateutil>=2.9
pytest>=8.0             # tests

pip install -r requirements.txt
```

3.3 API key management

Never hardcode API keys. Never commit them to git. Use a .env file for local development and environment variables in production:

```
# .env (add to .gitignore immediately)
ODDS_API_KEY=your_odds_api_key_here
BETFAIR_USERNAME=your_betfair_username
BETFAIR_PASSWORD=your_betfair_password
BETFAIR_APP_KEY=your_betfair_app_key
BETFAIR_CERT_PATH=/absolute/path/to/client-2048.crt
BETFAIR_CERT_KEY_PATH=/absolute/path/to/client-2048.key
TELEGRAM_BOT_TOKEN=123456:ABC-DEF...
TELEGRAM_CHAT_ID=your_chat_id

# config.py
import os
from dotenv import load_dotenv

load_dotenv()

ODDS_API_KEY = os.environ["ODDS_API_KEY"]
BETFAIR_USERNAME = os.environ["BETFAIR_USERNAME"]
BETFAIR_PASSWORD = os.environ["BETFAIR_PASSWORD"]
BETFAIR_APP_KEY = os.environ["BETFAIR_APP_KEY"]
BETFAIR_CERT = os.environ["BETFAIR_CERT_PATH"]
BETFAIR_CERT_KEY = os.environ["BETFAIR_CERT_KEY_PATH"]
TELEGRAM_BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
TELEGRAM_CHAT_ID = os.environ["TELEGRAM_CHAT_ID"]

# Detection thresholds
MIN_ARB_MARGIN = 0.005    # 0.5% floor
MAX_ARB_MARGIN = 0.05     # anything above 5% is suspicious
DEFAULT_STAKE = 200.0     # £, total per arb
SUSPECT_MARGIN = 0.04     # margin above which we flag as suspect
```

```
# Books we'll consider
BOOKS = {
    "pinnacle", "betfair_ex_uk", "smarkets", "matchbook",
    "bet365", "williamhill", "unibet", "paddypower",
    "coral", "ladbrokes", "marathonbet", "sport888",
}
```

4. Data Models

A well-designed model layer is the single biggest determinant of how maintainable the system will be. Use dataclasses (or Pydantic models) rather than dicts — you get IDE autocomplete, type checks, and a place to hang helper methods.

```
# models.py
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional

@dataclass(frozen=True, slots=True)
class Price:
    """A single bookmaker's price on a single outcome."""
    bookmaker: str
    decimal_odds: float
    last_update: datetime
    max_stake: Optional[float] = None # in-book limit if known

    @property
    def implied_prob(self) -> float:
        return 1.0 / self.decimal_odds

@dataclass(frozen=True, slots=True)
class Outcome:
    """One side of a market (e.g. 'Djokovic', 'Draw', 'Over 2.5')."""
    name: str
    prices: tuple[Price, ...]

    def best(self) -> Price:
        return max(self.prices, key=lambda p: p.decimal_odds)

@dataclass(frozen=True, slots=True)
class Market:
    """A market on an event: h2h, totals, spreads etc."""
    key: str # 'h2h', 'totals', 'spreads'
    outcomes: tuple[Outcome, ...]

@dataclass(frozen=True, slots=True)
class Event:
    id: str
    sport: str
    league: str
    home: str
    away: str
    commence_time: datetime
    markets: tuple[Market, ...]

    def market(self, key: str) -> Optional[Market]:
        return next((m for m in self.markets if m.key == key), None)

@dataclass(frozen=True, slots=True)
class ArbLeg:
    outcome_name: str
    bookmaker: str
    decimal_odds: float
    stake: float

    @property
    def payout(self) -> float:
        return self.stake * self.decimal_odds
```

```

@dataclass(frozen=True, slots=True)
class Arb:
    event: Event
    market_key: str
    legs: tuple[ArbLeg, ...]
    total_stake: float
    margin: float                # (1/B) - 1
    detected_at: datetime

    @property
    def profit(self) -> float:
        # Guaranteed profit: min payout minus total stake
        return min(l.payout for l in self.legs) - self.total_stake

    @property
    def is_suspect(self) -> bool:
        return self.margin > 0.04      # See Part I §5.3

    def summary(self) -> str:
        legs = " | ".join(
            f"{l.outcome_name}@{l.decimal_odds:.2f}({l.bookmaker}):£{l.stake:.2f}"
            for l in self.legs
        )
        return (f"[{self.margin*100:.2f}%] {self.event.home} v {self.event.away} "
                f"({self.market_key}) -> {legs} -> £{self.profit:.2f} profit")

```

A few design decisions worth flagging:

- **frozen=True.** Immutable models are safer in concurrent code and can be used as dictionary keys / set elements.
- **slots=True.** Saves memory when you hold thousands of Price objects.
- **Tuples not lists.** Enforces immutability of the collection itself.
- **No IO in the model.** Models are pure data. API calls belong in sources/, storage in storage.py.
- **Timezone-aware datetimes.** All timestamps are UTC. Convert to local only for display.

5. Fetching Odds — The Odds API

The Odds API (the-odds-api.com) is the simplest way to get started. Sign up for a free key (500 requests/month is enough for a demo but not for production). Endpoints you will use:

Every request consumes credits (1 for events, 1 per region-market combination for odds). Cache responses locally; the market moves in seconds but not milliseconds, and there is no point paying for a fresh call every scan.

5.1 The client

```
# sources/odds_api.py
from __future__ import annotations
import time
from datetime import datetime, timezone
from typing import Iterator

import requests
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception_type
from loguru import logger

from ..config import ODDS_API_KEY
from ..models import Event, Market, Outcome, Price

class OddsAPI:
    BASE = "https://api.the-odds-api.com/v4"

    def __init__(self, api_key: str = ODDS_API_KEY):
        self.api_key = api_key
        self.session = requests.Session()
        self._quota_remaining: int | None = None

    @retry(
        retry=retry_if_exception_type(requests.RequestException),
        stop=stop_after_attempt(4),
        wait=wait_exponential(multiplier=1, min=1, max=30),
        reraise=True,
    )
    def _get(self, path: str, **params) -> dict | list:
        params["apiKey"] = self.api_key
        url = f"{self.BASE}{path}"
        resp = self.session.get(url, params=params, timeout=10)
        if resp.status_code == 429:
            logger.warning("Odds API rate-limited; backing off")
            time.sleep(5)
            raise requests.RequestException("429")
        resp.raise_for_status()
        remaining = resp.headers.get("x-requests-remaining")
        if remaining is not None:
            self._quota_remaining = int(remaining)
            logger.debug(f"Odds API credits remaining: {remaining}")
        return resp.json()

    def sports(self) -> list[dict]:
        return self._get("/sports")

    def odds(
        self,
        sport: str,
        regions: str = "uk,eu",
        markets: str = "h2h",
        odds_format: str = "decimal",
    ) -> Iterator[Event]:
        raw = self._get(
```

```

        f"/sports/{sport}/odds",
        regions=regions,
        markets=markets,
        oddsFormat=odds_format,
        dateFormat="iso",
    )
    for ev in raw:
        yield self._parse_event(ev, sport)

def _parse_event(self, raw: dict, sport: str) -> Event:
    markets_by_key: dict[str, list[tuple[str, Price]]] = {}
    for book in raw.get("bookmakers", []):
        book_key = book["key"]
        last = datetime.fromisoformat(book["last_update"].replace("Z", "+00:00"))
        for mkt in book.get("markets", []):
            mkey = mkt["key"]
            markets_by_key.setdefault(mkey, [])
            for oc in mkt.get("outcomes", []):
                price = Price(
                    bookmaker=book_key,
                    decimal_odds=float(oc["price"]),
                    last_update=last,
                )
                markets_by_key[mkey].append((oc["name"], price))

    markets: list[Market] = []
    for mkey, entries in markets_by_key.items():
        by_outcome: dict[str, list[Price]] = {}
        for name, price in entries:
            by_outcome.setdefault(name, []).append(price)
        outcomes = tuple(
            Outcome(name=n, prices=tuple(ps)) for n, ps in by_outcome.items()
        )
        markets.append(Market(key=mkey, outcomes=outcomes))

    return Event(
        id=raw["id"],
        sport=sport,
        league=raw.get("sport_title", sport),
        home=raw["home_team"],
        away=raw["away_team"],
        commence_time=datetime.fromisoformat(
            raw["commence_time"].replace("Z", "+00:00")
        ),
        markets=tuple(markets),
    )

```

A few points worth noting:

- **Retries with exponential backoff** via tenacity. Network calls fail; plan for it.
- **Quota tracking** from the response headers. Log the remaining credits so you can catch runaway loops before they exhaust your monthly budget.
- **Streaming parse** — yield events one at a time so the caller can process incrementally rather than waiting for a full list to build.
- **Timezone normalisation** at the boundary. Everything internal is UTC.

6. Normalising Across Bookmakers

The Odds API already reconciles bookmakers into a common structure for you — different books' odds for the same event arrive in the same JSON blob. This is the killer feature of an aggregator. When you graduate to hitting bookmaker APIs directly (or scraping — legally grey), you inherit the hard problem: two books' APIs will disagree on almost every string.

6.1 Team and player name reconciliation

Even within a single aggregator, names occasionally drift: 'Manchester City' at one book, 'Man City' at another. When merging feeds, use fuzzy matching:

```
# normalise.py
from rapidfuzz import fuzz, process

def match_team(name: str, candidates: list[str], threshold: int = 85) -> str | None:
    """Return the best-matching candidate, or None if below threshold."""
    result = process.extractOne(
        name, candidates,
        scorer=fuzz.WRatio,
        score_cutoff=threshold,
    )
    return result[0] if result else None

# Alias table for known-difficult cases
TEAM_ALIASES = {
    "Man City": "Manchester City",
    "Man Utd": "Manchester United",
    "Wolves": "Wolverhampton Wanderers",
    "PSG": "Paris Saint Germain",
    "Bayern": "Bayern Munich",
    # ... build this up as you encounter mismatches
}

def canonical_team(name: str) -> str:
    return TEAM_ALIASES.get(name.strip(), name.strip())
```

6.2 Market key reconciliation

Different sources use different keys for the same market:

Maintain a central mapping module:

```
# normalise.py (continued)
MARKET_CANONICAL = {
    # Odds API keys
    "h2h": "h2h",
    "totals": "totals",
    "spreads": "spreads",
    # Betfair keys
    "MATCH_ODDS": "h2h",
    "OVER_UNDER_25": "totals_2.5",
    "ASIAN_HANDICAP": "spreads",
    # Add more as encountered
}

def canonical_market(source_key: str) -> str:
    if source_key not in MARKET_CANONICAL:
        raise KeyError(f"Unknown market key: {source_key}")
    return MARKET_CANONICAL[source_key]
```

Warning. Silent mismatches are the number-one source of false-positive arbs. If your normalisation function returns 'totals' for both 'over 2.5' at Book A and 'over 2.75' at Book B, you will detect arbs that do not exist and lose real money placing them. Include the line value in the market key: 'totals_2.5' not just 'totals'.

7. The Arbitrage Detection Algorithm

Given a normalised event with a market and multiple prices per outcome, arb detection is straightforward: pick the best price for each outcome, sum the implied probabilities, and check whether the total is below 1.

7.1 Two-way and three-way case

```
# detector.py
from datetime import datetime, timezone
from typing import Iterable, Iterator

from .models import Event, Market, Arb, ArbLeg
from .config import MIN_ARB_MARGIN, MAX_ARB_MARGIN, DEFAULT_STAKE

def detect_arb(event: Event, market: Market,
               total_stake: float = DEFAULT_STAKE,
               books_allowed: set[str] | None = None
               ) -> Arb | None:
    """Return an Arb for this market if one exists, else None.
    Considers only outcomes in `market` and best prices across `books_allowed`.
    """
    best_prices = []
    for outcome in market.outcomes:
        prices = outcome.prices
        if books_allowed is not None:
            prices = tuple(p for p in prices if p.bookmaker in books_allowed)
        if not prices:
            return None # no coverage for this leg
        best = max(prices, key=lambda p: p.decimal_odds)
        best_prices.append((outcome.name, best))

    B = sum(1.0 / price.decimal_odds for _, price in best_prices)
    if B >= 1.0:
        return None # no arb
    margin = (1.0 / B) - 1.0
    if not (MIN_ARB_MARGIN <= margin <= MAX_ARB_MARGIN):
        return None # below floor or suspicious

    legs = tuple(
        ArbLeg(
            outcome_name=name,
            bookmaker=price.bookmaker,
            decimal_odds=price.decimal_odds,
            stake=round(total_stake * (1.0 / price.decimal_odds) / B, 2),
        )
        for name, price in best_prices
    )

    return Arb(
        event=event,
        market_key=market.key,
        legs=legs,
        total_stake=total_stake,
        margin=margin,
        detected_at=datetime.now(timezone.utc),
    )

def scan_events(events: Iterable[Event],
                markets: set[str] = frozenset({"h2h"}),
                books_allowed: set[str] | None = None
                ) -> Iterator[Arb]:
    for ev in events:
        for m in ev.markets:
```

```

if m.key not in markets:
    continue
arb = detect_arb(ev, m, books_allowed=books_allowed)
if arb is not None:
    yield arb

```

7.2 Complexity analysis

For n outcomes each with k prices, finding the best price per outcome is $O(nk)$. Summing is $O(n)$. Total per market: $O(nk)$. For a typical scan across a few thousand events, this is trivial — well under 100 ms. Optimisation is not needed; correctness is.

7.3 Excluding bookmakers you cannot use

The `books_allowed` parameter is critical. There is no point detecting arbs at bookmakers where you have no account, no balance, or a limited stake. Maintain a per-book state file listing the maximum stake you can place at each and filter accordingly.

7.4 Testing the detector

```

# tests/test_detector.py
from datetime import datetime, timezone
from arbengine.models import Event, Market, Outcome, Price
from arbengine.detector import detect_arb

NOW = datetime.now(timezone.utc)

def make_price(book: str, odds: float) -> Price:
    return Price(bookmaker=book, decimal_odds=odds, last_update=NOW)

def test_two_way_arb():
    market = Market(key="h2h", outcomes=(
        Outcome(name="A", prices=(make_price("bookA", 2.10),)),
        Outcome(name="B", prices=(make_price("bookB", 2.10),)),
    ))
    event = Event(id="e1", sport="tennis", league="ATP",
        home="A", away="B", commence_time=NOW, markets=(market,))
    arb = detect_arb(event, market, total_stake=1000.0)
    assert arb is not None
    # implied prob 1/2.10 + 1/2.10 = 0.9524, margin ~ 5%
    assert abs(arb.margin - 0.05) < 0.001
    # equal profit ~ £50
    assert abs(arb.profit - 50.0) < 1.0

def test_no_arb_when_book_gt_1():
    market = Market(key="h2h", outcomes=(
        Outcome(name="A", prices=(make_price("bookA", 1.90),)),
        Outcome(name="B", prices=(make_price("bookB", 1.90),)),
    ))
    event = Event(id="e2", sport="tennis", league="ATP",
        home="A", away="B", commence_time=NOW, markets=(market,))
    assert detect_arb(event, market) is None

def test_suspiciously_high_margin_rejected():
    # An 'arb' at 20% margin is almost certainly a bug or palp error.
    market = Market(key="h2h", outcomes=(
        Outcome(name="A", prices=(make_price("bookA", 3.00),)),
        Outcome(name="B", prices=(make_price("bookB", 3.00),)),
    ))
    event = Event(id="e3", sport="tennis", league="ATP",
        home="A", away="B", commence_time=NOW, markets=(market,))

```

```
assert detect_arb(event, market) is None
```

8. The Stake Calculator

The stake sizing is embedded in `detect_arb()` above but there are refinements you will want in production.

8.1 Handling minimum stake constraints

```
# sizing.py
from math import ceil, floor

from .models import Arb, ArbLeg

def round_stakes(arb: Arb,
                 step: float = 0.01,
                 min_stake_per_leg: float = 1.0
                 ) -> Arb:
    """Round each leg's stake to a multiple of `step`, respecting minimum.

    Rounds down; the total staked may be less than arb.total_stake and the
    guaranteed profit is recomputed. If any leg falls below min_stake_per_leg,
    returns the original (unadjusted) arb with a warning attribute – caller
    should skip.
    """
    new_legs = []
    for leg in arb.legs:
        rounded = floor(leg.stake / step) * step
        if rounded < min_stake_per_leg:
            # Cannot meet minimum on some leg; give up.
            return arb
        new_legs.append(ArbLeg(
            outcome_name=leg.outcome_name,
            bookmaker=leg.bookmaker,
            decimal_odds=leg.decimal_odds,
            stake=round(rounded, 2),
        ))
    total = sum(l.stake for l in new_legs)
    return Arb(
        event=arb.event,
        market_key=arb.market_key,
        legs=tuple(new_legs),
        total_stake=total,
        margin=arb.margin,          # margin per-unit-turnover unchanged
        detected_at=arb.detected_at,
    )
```

8.2 Handling per-book maximum stakes

If your account at Bet365 is limited to £50 maximum stake, and the equal-profit calculation asks for £200 on that leg, you must scale the entire arb down. The constraining leg determines total stake:

```
# sizing.py (continued)

def apply_book_limits(arb: Arb, limits: dict[str, float]) -> Arb | None:
    """Scale the whole arb so no leg exceeds its book limit.

    Returns None if the resulting total stake is too small to be worthwhile.
    """
    scale_factors = []
    for leg in arb.legs:
        cap = limits.get(leg.bookmaker)
        if cap is None:
            continue
        if leg.stake > cap:
            scale_factors.append(cap / leg.stake)
```

```

if not scale_factors:
    return arb

scale = min(scale_factors)
new_legs = tuple(
    ArbLeg(
        outcome_name=l.outcome_name,
        bookmaker=l.bookmaker,
        decimal_odds=l.decimal_odds,
        stake=round(l.stake * scale, 2),
    )
    for l in arb.legs
)
new_total = sum(l.stake for l in new_legs)
if new_total < 20.0:      # not worth placing five bets for pennies
    return None
return Arb(
    event=arb.event,
    market_key=arb.market_key,
    legs=new_legs,
    total_stake=new_total,
    margin=arb.margin,
    detected_at=arb.detected_at,
)

```

8.3 Rounding to psychology-safe stakes

If you are placing bets manually at soft books, exact-arithmetic stakes (£47.83, £52.17) scream 'arber'. Rounding to £45 / £55 leaves a tiny bit of directional exposure but is much less conspicuous. This is a real design decision:

```

def round_to_conspicuously_recreational(arb: Arb, step: int = 5) -> Arb:
    """Round stakes to £5 multiples. Introduces directional exposure!"""
    new_legs = tuple(
        ArbLeg(
            outcome_name=l.outcome_name,
            bookmaker=l.bookmaker,
            decimal_odds=l.decimal_odds,
            stake=float(round(l.stake / step) * step),
        )
        for l in arb.legs
    )
    total = sum(l.stake for l in new_legs)
    # Recompute payouts explicitly – profit is now min payout - total
    return Arb(
        event=arb.event,
        market_key=arb.market_key,
        legs=new_legs,
        total_stake=total,
        margin=arb.margin,
        detected_at=arb.detected_at,
    )

```

Practical note. The margin field of Arb stores the theoretical per-turnover margin. Once you round stakes, the actual profit varies by outcome. Always compute worst-case profit explicitly before committing to a bet.

9. The Full Scanner Pipeline

Wire the components together. The scanner is a loop that polls sources, detects arbs, sizes stakes, dedupes against previously-alerted opportunities, and emits alerts.

```
# scanner.py
from __future__ import annotations
import time
from datetime import datetime, timezone, timedelta
from typing import Iterable

from loguru import logger

from .config import BOOKS, DEFAULT_STAKE
from .sources.odds_api import OddsAPI
from .detector import scan_events
from .sizing import apply_book_limits, round_stakes
from .storage import ArbStore
from .alerts import Telegram
from .models import Arb

class Scanner:
    def __init__(self,
                 sports: list[str],
                 markets: set[str] = frozenset({"h2h"}),
                 book_limits: dict[str, float] | None = None,
                 poll_interval: int = 30,
                 dedup_window_s: int = 300):
        self.sports = sports
        self.markets = markets
        self.book_limits = book_limits or {}
        self.poll_interval = poll_interval
        self.dedup_window = timedelta(seconds=dedup_window_s)
        self.odds = OddsAPI()
        self.store = ArbStore("arbs.db")
        self.tg = Telegram()
        self._seen: dict[str, datetime] = {}

    def _key(self, arb: Arb) -> str:
        legs = "|".join(f"{l.bookmaker}:{l.outcome_name}:{l.decimal_odds:.2f}"
                        for l in arb.legs)
        return f"{arb.event.id}|{arb.market_key}|{legs}"

    def _is_new(self, arb: Arb) -> bool:
        k = self._key(arb)
        now = arb.detected_at
        prev = self._seen.get(k)
        if prev is not None and now - prev < self.dedup_window:
            return False
        self._seen[k] = now
        # prune old
        cutoff = now - self.dedup_window
        self._seen = {kk: vv for kk, vv in self._seen.items() if vv > cutoff}
        return True

    def scan_once(self) -> list[Arb]:
        found = []
        for sport in self.sports:
            try:
                events = list(self.odds.odds(sport, regions="uk,eu",
                                             markets=".".join(self.markets)))
            except Exception as e:
                logger.exception(f"Error fetching {sport}: {e}")
                continue

            for arb in scan_events(events, markets=self.markets, books_allowed=BOOKS):
                if self._is_new(arb):
                    found.append(arb)
                    self.tg.send(arb)
                    self.store.save(arb)
```

```

        arb = apply_book_limits(arb, self.book_limits)
        if arb is None:
            continue
        arb = round_stakes(arb)
        if not self._is_new(arb):
            continue
        found.append(arb)
        self.store.save(arb)
        self.tg.send(arb.summary())
        logger.success(arb.summary())
    return found

def run_forever(self):
    logger.info(f"Scanner starting; sports={self.sports}, "
               f"markets={self.markets}, interval={self.poll_interval}s")
    while True:
        start = time.monotonic()
        n = len(self.scan_once())
        elapsed = time.monotonic() - start
        logger.info(f"Scan complete: {n} arbs found in {elapsed:.1f}s")
        sleep = max(1.0, self.poll_interval - elapsed)
        time.sleep(sleep)

```

9.1 Entry point

```

# main.py
from loguru import logger

from arbengine.scanner import Scanner

BOOK_LIMITS = {
    "bet365": 50.0,          # you've been factored down
    "williamhill": 100.0,
    "unibet": 200.0,
    "pinnacle": 5000.0,
    "betfair_ex_uk": 10000.0,
    # ...
}

if __name__ == "__main__":
    logger.add("scanner.log", rotation="10 MB")
    scanner = Scanner(
        sports=[
            "soccer_epl", "soccer_uefa_champs_league",
            "tennis_atp_singles", "tennis_wta_singles",
            "basketball_nba", "americanfootball_nfl",
        ],
        markets={"h2h"},
        book_limits=BOOK_LIMITS,
        poll_interval=30,
    )
    scanner.run_forever()

```

10. Alerting via Telegram

For arbs you place manually, a phone push notification within a second of detection is the difference between catching the arb and watching it evaporate. Telegram bots are the standard tool: free, fast, well-documented, and unlike email do not get lost in spam.

10.1 Setting up a bot

Open Telegram, message @BotFather, run /newbot, follow the prompts. You get a bot token like 123456:ABC-DEF1234ghIk1-zyx57W2v1u123ew11. Add the bot to a chat with yourself. Get the chat_id by messaging the bot once and calling getUpdates:

```
curl https://api.telegram.org/bot<TOKEN>/getUpdates | jq
# Look for message.chat.id in the response.
```

10.2 The alerter

```
# alerts.py
from __future__ import annotations
import time
from typing import Optional

import requests
from loguru import logger

from .config import TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID

class Telegram:
    BASE = "https://api.telegram.org"

    def __init__(self,
                 bot_token: str = TELEGRAM_BOT_TOKEN,
                 chat_id: str = TELEGRAM_CHAT_ID):
        self.bot_token = bot_token
        self.chat_id = chat_id
        self._last_sent: float = 0.0

    def send(self, text: str, silent: bool = False) -> Optional[dict]:
        # Telegram rate limit: 30 msgs/sec globally, 1/sec per chat.
        now = time.time()
        wait = 1.05 - (now - self._last_sent)
        if wait > 0:
            time.sleep(wait)
        self._last_sent = time.time()

        url = f"{self.BASE}/bot{self.bot_token}/sendMessage"
        try:
            r = requests.post(url, timeout=5, json={
                "chat_id": self.chat_id,
                "text": text,
                "disable_notification": silent,
                "parse_mode": "HTML",
            })
            r.raise_for_status()
            return r.json()
        except Exception as e:
            logger.warning(f"Telegram send failed: {e}")
            return None

    def send_arb(self, arb) -> None:
        legs_html = "\n".join(
            f" \u2022 <b>{l.outcome_name}</b> @ <code>{l.decimal_odds:.2f}</code> "
            f"({l.bookmaker}) \u2022 {l.stake:.2f}"
            for l in arb.legs
        )
```

```

)
text = (
    f"<b>ARB {arb.margin*100:.2f}%</b> – "
    f"{arb.event.home} v {arb.event.away}\n"
    f"<i>{arb.event.league}, {arb.market_key}</i>\n"
    f"{legs_html}\n"
    f"Total stake £{arb.total_stake:.2f} \u2192 profit £{arb.profit:.2f}\n"
    f"KO: {arb.event.commence_time:%Y-%m-%d %H:%M} UTC"
)
self.send(text)

```

11. The Betfair Exchange API

The Betfair Exchange API is the workhorse of programmatic UK sports betting. Two components matter for our purposes:

- **Betting API** — read market data, place bets, cancel bets, list current orders.
- **Accounts API** — session management, account balance, transaction history.

Betfair supports two authentication modes. Interactive is for humans; non-interactive (certificate-based) is for automated systems and is what you need.

11.1 Prerequisites

- A funded Betfair account.
- An Application Key — obtain via the API-NG accounts endpoint (one-off £/€10 fee).
- A client certificate pair (client-2048.crt and client-2048.key) uploaded to your account.
- Two-factor authentication temporarily disabled while you generate the cert, then re-enabled.

Full setup instructions are at developer.betfair.com. Do not skip the certificate step; it is what enables unattended login.

11.2 The client (using `betfairlightweight`)

The community-maintained `betfairlightweight` library wraps Betfair's JSON-RPC endpoints in an ergonomic Python interface.

```
# sources/betfair.py
from __future__ import annotations
from datetime import datetime, timezone
from typing import Iterator

import betfairlightweight as bfl
from betfairlightweight.filters import (
    market_filter, price_projection, price_data, place_instruction, limit_order,
)
from loguru import logger

from ..config import (
    BETFAIR_USERNAME, BETFAIR_PASSWORD, BETFAIR_APP_KEY,
    BETFAIR_CERT, BETFAIR_CERT_KEY,
)
from ..models import Event, Market, Outcome, Price

class Betfair:
    def __init__(self):
        self.client = bfl.APIClient(
            username=BETFAIR_USERNAME,
            password=BETFAIR_PASSWORD,
            app_key=BETFAIR_APP_KEY,
            certs=(BETFAIR_CERT, BETFAIR_CERT_KEY),
            lightweight=True,      # return dicts not model objects
        )
        self._logged_in = False

    def login(self) -> None:
        if not self._logged_in:
            self.client.login()
            self._logged_in = True
            logger.info("Betfair: authenticated")
```

```

def logout(self) -> None:
    if self._logged_in:
        self.client.logout()
        self._logged_in = False

def event_types(self) -> list[dict]:
    return self.client.betting.list_event_types(filter=market_filter())

def list_markets(self,
                 event_type_ids: list[str],
                 country_codes: list[str] | None = None,
                 hours_ahead: int = 24
                 ) -> list[dict]:
    """Return a list of MarketCatalogue dicts."""
    self.login()
    mf = market_filter(
        event_type_ids=event_type_ids,
        market_countries=country_codes,
        market_type_codes=["MATCH_ODDS"],
        market_start_time={
            "from": datetime.now(timezone.utc).isoformat(),
            "to": (datetime.now(timezone.utc)
                  .replace(microsecond=0)
                  .isoformat()),
        },
    )
    return self.client.betting.list_market_catalogue(
        filter=mf,
        max_results="100",
        market_projection=["EVENT", "RUNNER_DESCRIPTION", "MARKET_START_TIME"],
    )

def market_book(self, market_ids: list[str]) -> list[dict]:
    self.login()
    return self.client.betting.list_market_book(
        market_ids=market_ids,
        price_projection=price_projection(
            price_data=price_data(ex_best_offers=True),
        ),
    )

```

11.3 What Betfair returns

A MarketBook contains 'runners' — each an outcome — and each runner has an 'ex' (exchange) block with the current best-back and best-lay stacks:

```

# Example return from market_book, single runner (Djokovic to win):
{
  "marketId": "1.234567890",
  "runners": [
    {
      "selectionId": 12345,
      "runnerName": "Djokovic",
      "status": "ACTIVE",
      "handicap": 0.0,
      "ex": {
        "availableToBack": [
          {"price": 2.10, "size": 234.50},    # best back price
          {"price": 2.08, "size": 512.30},
          {"price": 2.06, "size": 1200.00},
        ],
        "availableToLay": [
          {"price": 2.12, "size": 189.20},    # best lay price
          {"price": 2.14, "size": 502.10},
          {"price": 2.16, "size": 850.00},
        ]
      }
    }
  ],
}

```

```
        # ... other runners  
    ]  
}
```

The **back price** is the odds at which you can bet *on* the selection; the **lay price** is the odds at which you can bet *against*. The two-tick gap between best-back and best-lay is the exchange's implicit spread — analogous to bid-ask in equities.

12. Placing Bets Programmatically

This is the core of automated execution. Betfair's `place_orders` endpoint takes a list of instructions and returns a status per instruction. Always place with limit orders — never market orders on an exchange, because the top of book can be a single £10 offer with a wide gap behind it.

12.1 Placing a back bet

```
# executor.py
from __future__ import annotations
from decimal import Decimal
from typing import Literal

from betfairlightweight.filters import place_instruction, limit_order
from loguru import logger

from .sources.betfair import Betfair

PersistenceType = Literal["LAPSE", "PERSIST", "MARKET_ON_CLOSE"]

class BetfairExecutor:
    def __init__(self, bf: Betfair):
        self.bf = bf

    def place_back(self,
                   market_id: str,
                   selection_id: int,
                   price: float,
                   size: float,
                   persistence: PersistenceType = "LAPSE",
                   customer_ref: str | None = None
                   ) -> dict:
        """Place a single BACK limit order and return the response.

        `price` is in Betfair-legal increments; use round_to_tick() below first.
        `size` is stake in the account's currency.
        `persistence` LAPSE means unmatched portion is cancelled at kick-off.
        """
        self.bf.login()
        instruction = place_instruction(
            order_type="LIMIT",
            selection_id=selection_id,
            side="BACK",
            limit_order=limit_order(
                size=round(size, 2),
                price=price,
                persistence_type=persistence,
            ),
        )
        resp = self.bf.client.betting.place_orders(
            market_id=market_id,
            instructions=[instruction],
            customer_ref=customer_ref,
        )
        logger.info(f"place_back mid={market_id} sel={selection_id} "
                    f"{size}@{price} -> {resp['status']}")
        return resp

    def place_lay(self,
                  market_id: str,
                  selection_id: int,
                  price: float,
                  size: float,
                  persistence: PersistenceType = "LAPSE",
```

```

        customer_ref: str | None = None
    ) -> dict:
        """Place a single LAY limit order.

        `size` here is the BACKER's stake – your liability is size*(price-1).
        """
        self.bf.login()
        instruction = place_instruction(
            order_type="LIMIT",
            selection_id=selection_id,
            side="LAY",
            limit_order=limit_order(
                size=round(size, 2),
                price=price,
                persistence_type=persistence,
            ),
        )
        resp = self.bf.client.betting.place_orders(
            market_id=market_id,
            instructions=[instruction],
            customer_ref=customer_ref,
        )
        logger.info(f"place_lay mid={market_id} sel={selection_id} "
                    f"{size}@{price} -> {resp['status']}")
        return resp

```

12.2 Betfair tick sizes

Betfair enforces price-tick rules. You cannot back at 2.11; you must back at 2.10 or 2.12. The tick size depends on the odds range:

```
# executor.py (continued)
```

```

BETFAIR_TICKS = [
    (2.00, 0.01),
    (3.00, 0.02),
    (4.00, 0.05),
    (6.00, 0.10),
    (10.00, 0.20),
    (20.00, 0.50),
    (30.00, 1.00),
    (50.00, 2.00),
    (100.00, 5.00),
    (1000.0, 10.0),
]

```

```

def round_to_tick(price: float, direction: str = "down") -> float:
    """Round `price` to nearest legal Betfair tick.
    direction='down' rounds towards 1.01 (conservative for a BACK).
    direction='up' rounds away from 1.01 (conservative for a LAY).
    """
    for upper, step in BETFAIR_TICKS:
        if price < upper:
            if direction == "down":
                return round(int(price / step) * step, 2)
            return round((int(price / step) + 1) * step, 2)
    return price

```

Warning. Placing a limit order at a price better than the top of book will not match; you sit on the queue. Placing at or through the best-back (for a back) or best-lay (for a lay) matches immediately for the available size, with the remainder resting on the queue. For arb hedging you want immediate execution, so place a back at (current best-back) and a lay at (current best-lay), accepting the tighter effective odds. If you want size, walk deeper into the book.

13. Order Management and Status Tracking

13.1 Reading order status

```
# executor.py (continued)
def list_current_orders(self, market_ids: list[str] | None = None) -> list[dict]:
    self.bf.login()
    kwargs = {}
    if market_ids:
        kwargs["market_ids"] = market_ids
    resp = self.bf.client.betting.list_current_orders(**kwargs)
    return resp.get("currentOrders", [])

def is_fully_matched(self, bet_id: str) -> bool:
    orders = self.list_current_orders()
    for o in orders:
        if o["betId"] == bet_id:
            return o["sizeRemaining"] == 0.0
    # Not in current orders -> already settled or fully matched and cleared
    return True

def cancel(self, market_id: str, bet_id: str, size_reduction: float | None = None):
    self.bf.login()
    instruction = {"betId": bet_id}
    if size_reduction is not None:
        instruction["sizeReduction"] = size_reduction
    return self.bf.client.betting.cancel_orders(
        market_id=market_id, instructions=[instruction],
    )
```

13.2 The two-leg race condition

The fundamental problem in arb execution: between placing leg 1 and leg 2, prices on leg 2 can move. Three strategies:

Fire both legs concurrently. Use asyncio and place both instructions in parallel. Accepts the risk that both might partially fill at worse prices, but minimises the window of directional exposure.

Leg 1 first, then leg 2 if it fills. Safer if leg 1 is likely to fill (deep book, sharp side). If leg 2 has moved, cancel leg 1's unmatched portion and eat the small loss on the matched portion. Suitable when one leg is illiquid.

Lay first, then back. Since exchange lay prices reflect real capital, and soft book back prices are quoted by the bookie's engine, the soft leg is more likely to be still available when you come back to it. Standard practice for matched-betting-style arbs.

```
# executor.py (continued)
import asyncio

async def _place_async(self, coro):
    return await asyncio.to_thread(lambda: coro())

async def place_arb_atomic(self, arb, hedge_selection_id: int, hedge_market_id: str):
    """Place a two-leg arb where one leg is on Betfair (lay) and the other is
    at a soft book (manual). Fires the Betfair lay immediately, and returns
    the details for the human to place the back leg fast.

    NOTE: Betfair-to-Betfair arbs across markets can be fully automated;
    Betfair-to-soft-bookie CANNOT be legally automated at the soft leg.
    """
```

```

# 1. Place the Betfair hedge (fast, deterministic pricing)
hedge_leg = next(l for l in arb.legs if l.bookmaker.startswith("betfair"))
resp = await asyncio.to_thread(
    self.place_layer,
    hedge_market_id, hedge_selection_id,
    hedge_leg.decimal_odds, hedge_leg.stake,
)
if resp["status"] != "SUCCESS":
    logger.error(f"Hedge leg failed: {resp}")
    return None

# 2. Alert the human to place the soft-book leg immediately
soft_leg = next(l for l in arb.legs if not l.bookmaker.startswith("betfair"))
self.bf._telegram.send(
    f"■ PLACE NOW at {soft_leg.bookmaker}: BACK {soft_leg.outcome_name} "
    f"£{soft_leg.stake:.2f} at min odds {soft_leg.decimal_odds:.2f}"
)
return resp

```

14. Persistence and Logging

Every arb detected, every bet placed, every settlement — persist it all. The value is in three places: reconciling expected vs realised P&L, quantifying void rates per book/sport/market, and debugging your detector when it produces false positives.

14.1 Schema

```
# storage.py
from __future__ import annotations
import json
import sqlite3
from datetime import datetime
from pathlib import Path

from .models import Arb

SCHEMA = """
CREATE TABLE IF NOT EXISTS arbs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    detected_at TEXT NOT NULL,
    event_id TEXT NOT NULL,
    sport TEXT NOT NULL,
    league TEXT NOT NULL,
    home TEXT NOT NULL,
    away TEXT NOT NULL,
    commence_time TEXT NOT NULL,
    market_key TEXT NOT NULL,
    margin REAL NOT NULL,
    total_stake REAL NOT NULL,
    profit REAL NOT NULL,
    legs_json TEXT NOT NULL,
    placed INTEGER NOT NULL DEFAULT 0,
    settled INTEGER NOT NULL DEFAULT 0,
    realised_pnl REAL
);

CREATE INDEX IF NOT EXISTS ix_arbs_detected ON arbs(detected_at);
CREATE INDEX IF NOT EXISTS ix_arbs_event ON arbs(event_id, market_key);

CREATE TABLE IF NOT EXISTS placements (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    arb_id INTEGER NOT NULL REFERENCES arbs(id),
    bookmaker TEXT NOT NULL,
    outcome_name TEXT NOT NULL,
    requested_odds REAL NOT NULL,
    requested_stake REAL NOT NULL,
    executed_odds REAL,
    executed_stake REAL,
    bet_id TEXT,
    status TEXT NOT NULL,
    placed_at TEXT NOT NULL
);
"""

class ArbStore:
    def __init__(self, path: str):
        self.path = Path(path)
        self.conn = sqlite3.connect(str(self.path))
        self.conn.executescript(SCHEMA)
        self.conn.commit()

    def save(self, arb: Arb) -> int:
        legs = [
```

```

        {"bookmaker": l.bookmaker, "outcome_name": l.outcome_name,
         "decimal_odds": l.decimal_odds, "stake": l.stake}
    for l in arb.legs
]
cur = self.conn.execute(
    """INSERT INTO arbs
        (detected_at, event_id, sport, league, home, away, commence_time,
         market_key, margin, total_stake, profit, legs_json)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
    (arb.detected_at.isoformat(), arb.event.id, arb.event.sport,
     arb.event.league, arb.event.home, arb.event.away,
     arb.event.commence_time.isoformat(),
     arb.market_key, arb.margin, arb.total_stake, arb.profit,
     json.dumps(legs)),
)
self.conn.commit()
return cur.lastrowid

def record_placement(self, arb_id: int, leg_book: str, leg_name: str,
                    req_odds: float, req_stake: float,
                    exec_odds: float | None, exec_stake: float | None,
                    bet_id: str | None, status: str) -> None:
    self.conn.execute(
        """INSERT INTO placements
            (arb_id, bookmaker, outcome_name, requested_odds, requested_stake,
             executed_odds, executed_stake, bet_id, status, placed_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (arb_id, leg_book, leg_name, req_odds, req_stake,
         exec_odds, exec_stake, bet_id, status,
         datetime.utcnow().isoformat()),
    )
    self.conn.commit()

def unsettled_arbs(self) -> list[dict]:
    rows = self.conn.execute(
        "SELECT * FROM arbs WHERE placed=1 AND settled=0"
    ).fetchall()
    cols = [d[0] for d in self.conn.execute(
        "SELECT * FROM arbs LIMIT 0"
    ).description]
    return [dict(zip(cols, r)) for r in rows]

```

14.2 Structured logging

Use loguru for structured logs; ship them to a file for long-term review and to stdout for live monitoring. Log at DEBUG for individual price fetches, INFO for scan cycles and successful placements, WARNING for retries and price movements, ERROR for failed placements, CRITICAL for anything that requires human intervention.

15. Backtesting Framework

Historical odds data lets you estimate what you would have made had you been running the scanner over a past period. This is the fastest way to tune parameters (minimum margin threshold, book filter) without spending real money.

15.1 Sources of historical data

- **Football-Data.co.uk** — free historical football odds CSVs at the closing and match-time levels; enough for a first backtest.
- **Betfair Historical Data** — full time-and-sales exchange data by market. Free to download, in a Betfair-specific streaming format. The gold standard.
- **OddsPortal scrape** — legally grey, do so at your own risk.
- **Snapshot your own live scans** — the easiest option once your scanner is running. Cost: SQLite grows quickly. Trim after 30 days.

15.2 A minimal backtester

```
# backtest.py
from __future__ import annotations
import json
import sqlite3
from datetime import datetime
from statistics import mean, stdev

from .models import Arb

def backtest_scanner(db_path: str,
                    min_margin: float = 0.005,
                    max_margin: float = 0.05,
                    void_rate: float = 0.03,
                    void_loss: float = 0.30) -> dict:
    """Replay historical arbs and compute realised P&L under a void model.

    void_rate: fraction of legs voided.
    void_loss: fraction of that leg's stake lost when voided
                (net of what you recover on the other legs).
    """
    import random
    random.seed(42)

    conn = sqlite3.connect(db_path)
    rows = conn.execute(
        """SELECT margin, total_stake, profit FROM arbs
           WHERE margin BETWEEN ? AND ?""",
        (min_margin, max_margin),
    ).fetchall()

    if not rows:
        return {"n": 0}

    outcomes = []
    for margin, stake, profit in rows:
        # Simple model: void_rate chance the arb is broken, in which case
        # you lose void_loss * stake instead of gaining profit.
        if random.random() < void_rate:
            outcomes.append(-void_loss * stake)
        else:
            outcomes.append(profit)
```

```

return {
    "n": len(outcomes),
    "total_pnl": sum(outcomes),
    "mean_pnl_per_arb": mean(outcomes),
    "stdev_pnl_per_arb": stdev(outcomes) if len(outcomes) > 1 else 0.0,
    "avg_margin": mean(r[0] for r in rows),
    "total_turnover": sum(r[1] for r in rows),
    "yield_on_turnover": sum(outcomes) / sum(r[1] for r in rows),
}

if __name__ == "__main__":
    from pprint import pprint
    pprint(backtest_scanner("arbs.db"))

```

Realistically your backtest will show something like 40–70% of the naive expected profit once voids are modelled. Any tuning that pushes backtest yield much above 30% annualised on turnover is likely over-fitting; be sceptical.

16. Rate Limits, Retries, and Robustness

16.1 Rate limits by source

16.2 Retry policy

Tenacity is the standard library. Retry only idempotent operations — reads always, writes never (except cancellations, which are idempotent):

```
from tenacity import (
    retry, stop_after_attempt, wait_exponential,
    retry_if_exception_type, before_sleep_log,
)
import logging

logger = logging.getLogger(__name__)

@retry(
    retry=retry_if_exception_type((ConnectionError, TimeoutError)),
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=1, max=60),
    before_sleep=before_sleep_log(logger, logging.WARNING),
    reraise=True,
)
def fetch_odds(...):
    ...
```

Warning. Never retry a `place_orders` call blindly. Betfair `place_orders` IS idempotent when you pass a `customerRef` — a duplicate `customerRef` within a 24h window returns the original response rather than placing a duplicate bet. Always pass a `UUID`; never retry without one, or you may place the same bet multiple times.

16.3 Session renewal

```
# betfair.py (continued)
from betfairlightweight.exceptions import InvalidResponse

def _with_relogin(self, fn, *args, **kwargs):
    self.login()
    try:
        return fn(*args, **kwargs)
    except InvalidResponse as e:
        # Common cause: session expired
        if "INVALID_SESSION_INFORMATION" in str(e):
            logger.warning("Session expired; re-authenticating")
            self._logged_in = False
            self.login()
            return fn(*args, **kwargs)
        raise
```

16.4 Circuit breakers

If the scanner sees 20+ arbs in one minute at margins > 8%, something is very wrong. A team name has been misparsed, or a price is a palpable error, or two different lines are being compared. Halt automatically and page a human:

```
class CircuitBreaker:
    def __init__(self, threshold: int = 20, window_s: int = 60,
                 min_margin_for_alarm: float = 0.08):
        self.threshold = threshold
        self.window_s = window_s
```

```
self.min_margin = min_margin_for_alarm
self._events: list[datetime] = []

def record(self, arb) -> bool:
    """Return True if breaker has tripped."""
    if arb.margin < self.min_margin:
        return False
    now = datetime.utcnow()
    self._events = [t for t in self._events
                    if (now - t).total_seconds() < self.window_s]
    self._events.append(now)
    return len(self._events) >= self.threshold
```

17. Latency and Execution

Solo arbers can survive with 5–30 second latencies. Professional operations run under 200 ms. You will never be a professional operation; do not try. But knowing where the latency lives matters for tuning.

17.1 Where the milliseconds go

- Fetch: 100–800 ms per API call. Cache aggressively; batch across sports/markets in one call when the API supports it.
- Parse: 5–50 ms. Negligible unless you are doing string operations at scale.
- Detect: 1–20 ms for a full scan of a few thousand events. Not a bottleneck.
- Alert: 200–1000 ms to Telegram. Fire-and-forget in a background thread rather than blocking the scanner.
- Place: 200–2000 ms round trip to Betfair. This is where you actually lose the arb if you serialise the two legs.

17.2 Concurrent fetches

Use `httpx` or `aiohttp` to fetch multiple sports concurrently:

```
# sources/odds_api_async.py
import asyncio
import httpx

from ..config import ODDS_API_KEY

class AsyncOddsAPI:
    BASE = "https://api.the-odds-api.com/v4"

    def __init__(self):
        self._client = httpx.AsyncClient(timeout=10.0)

    async def close(self):
        await self._client.aclose()

    async def _get(self, path: str, **params) -> dict | list:
        params["apiKey"] = ODDS_API_KEY
        r = await self._client.get(f"{self.BASE}{path}", params=params)
        r.raise_for_status()
        return r.json()

    async def fetch_many(self, sports: list[str], regions: str = "uk,eu",
                        markets: str = "h2h") -> dict[str, list]:
        tasks = [
            self._get(f"/sports/{s}/odds",
                    regions=regions, markets=markets,
                    oddsFormat="decimal", dateFormat="iso")
            for s in sports
        ]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        return {
            s: (r if not isinstance(r, Exception) else [])
            for s, r in zip(sports, results)
        }
```

For 8 sports, this cuts wall-clock fetch time from ~5 seconds to ~800 ms — often the single biggest latency win.

17.3 Do not colocate; do not build a bot

Betfair specifically prohibits certain kinds of automated behaviour — betting bots attempting to front-run others, HFT-style order layering, exploitation of matched bet delays. Your arb execution is safe territory, but read the API terms carefully. In particular: **never scrape Betfair via HTML**. Use the API. Scraping their website is explicitly banned and can result in permanent account closure.

18. Deployment

18.1 Where to run it

The scanner needs to be always on. Options in rough order of cost:

- **Your laptop.** Fine for a few weeks of testing. Fails the moment you close the lid or move house.
- **A Raspberry Pi at home.** Works if your home internet is stable. £50 one-off, essentially zero running cost. Consider a small UPS.
- **A £5/month VPS** (Hetzner, DigitalOcean, Linode). Standard choice. Deploy with Docker Compose, monitor via UptimeRobot.
- **Cloud functions** (AWS Lambda, Cloud Run) triggered on a schedule. Overkill for an arb scanner; pay-per-invocation adds up.

18.2 Dockerfile

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc libssl-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY arbengine/ ./arbengine/
COPY certs/ ./certs/      # Betfair client cert (chmod 0400)

ENV PYTHONUNBUFFERED=1

CMD ["python", "-m", "arbengine.main"]

# docker-compose.yml
services:
  scanner:
    build: .
    restart: unless-stopped
    env_file: .env
    volumes:
      - ./data:/app/data
      - ./logs:/app/logs
    healthcheck:
      test: ["CMD", "python", "-c", "import os,sys; sys.exit(0 if os.path.exists('/app/data/heartbeat') else 1)"]
      interval: 60s
      timeout: 10s
      retries: 3
```

18.3 Monitoring

You want two independent monitors:

- **Process liveness.** Systemd or docker-compose restart on crash. UptimeRobot pinging an endpoint your scanner writes to on each successful cycle.
- **Business logic.** Alert if no arbs detected in 6 hours (something wrong with the feed), or if void rate exceeds 6% over a rolling week (either book behaviour changed or your normalisation is silently broken).

18.4 Secrets hygiene

- Never commit `.env` or client certificates to git. Add both to `.gitignore`.
- Rotate the Betfair app key annually.
- Do not share Telegram bot tokens; anyone with the token can impersonate the bot.
- Chmod certificate files to 0400 and own by the running user.

19. Legal and ToS Considerations

Warning. Read this section carefully. The technical setup is straightforward; the operational compliance is where beginners get themselves into real trouble.

19.1 What is allowed

- Automating detection of arb opportunities from any legal source.
- Placing bets programmatically on venues whose ToS permit it — Betfair Exchange, Smarkets, Matchbook (with some restrictions).
- Storing your own bet history and analysis privately.
- Sharing methodology (as in this document) without sharing identifying credentials.

19.2 What is not allowed

- Automating placement at bookmakers whose ToS prohibit it (all major UK soft books, including Bet365, William Hill, Coral, Ladbrokes, Paddy Power, and DraftKings/FanDuel in the US). Doing so will get accounts closed and balances confiscated.
- Scraping bookmaker websites in ways that circumvent their fraud controls.
- Sharing accounts, using multiple accounts, or using accounts registered to others. This is fraud, not a ToS matter.
- Any use of automated placement to exploit promotional offers in bad faith. Bonus abuse clauses are broad.
- Failing to pay tax on any income arising from the operation of a betting business (as distinct from personal winnings, which in the UK are untaxed for the bettor).

19.3 The Betfair-specific rules

Betfair's Automated Betting Terms allow API betting but require:

- Use of a registered application key linked to a specific account.
- No sharing of the key with third parties.
- No use of the API to gain 'unfair advantage' over other users — this is broad and vague but has been interpreted against order-book manipulation and mass unmatched-order layering to disrupt price discovery.
- Compliance with rate limits.

Reading arb prices and placing a hedge lay in response to a soft-book back is well within acceptable use. HFT-style order layering is not.

19.4 Money laundering / tax

As covered in Part I §14: UK personal betting winnings are not taxable, but running a betting operation as a trade can be. Where the line falls is unclear and would depend on your specific circumstances; if you find yourself building a serious operation (withdrawing £50k+ per year, employing anyone, systematic account management), consult an accountant familiar with gambling law before receiving your first meaningful distribution.

For AML purposes, keep detailed records of every deposit, bet, and withdrawal. When your bank asks — and eventually your bank will ask — you want to be able to show the source of every pound in your account.

20. Project Layout and Next Steps

20.1 A complete minimal project

Here is the smallest viable arb-scanner-with-Telegram-alerts. It uses The Odds API for detection and does not attempt Betfair execution:

```
arb-minimal/
■■■ .env
■■■ .gitignore
■■■ requirements.txt
■■■ arb.py

# arb.py
import os, time, requests
from dotenv import load_dotenv
load_dotenv()

ODDS_KEY = os.environ["ODDS_API_KEY"]
TG_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
TG_CHAT = os.environ["TELEGRAM_CHAT_ID"]

MIN_MARGIN, MAX_MARGIN = 0.005, 0.05

def fetch(sport):
    r = requests.get(
        f"https://api.the-odds-api.com/v4/sports/{sport}/odds",
        params={"apiKey": ODDS_KEY, "regions": "uk,eu",
                "markets": "h2h", "oddsFormat": "decimal"},
        timeout=10)
    r.raise_for_status()
    return r.json()

def find_arb(event):
    # {outcome_name: (odds, bookmaker)}
    best = {}
    for book in event["bookmakers"]:
        for mkt in book["markets"]:
            if mkt["key"] != "h2h":
                continue
            for oc in mkt["outcomes"]:
                cur = best.get(oc["name"])
                if cur is None or oc["price"] > cur[0]:
                    best[oc["name"]] = (oc["price"], book["key"])
    if len(best) < 2:
        return None
    B = sum(1/o for o, _ in best.values())
    if not MIN_MARGIN <= (1/B - 1) <= MAX_MARGIN:
        return None
    return B, best

def notify(text):
    requests.post(
        f"https://api.telegram.org/bot{TG_TOKEN}/sendMessage",
        json={"chat_id": TG_CHAT, "text": text}, timeout=5)

SPORTS = ["soccer_epl", "tennis_atp_singles", "basketball_nba"]

while True:
    for sport in SPORTS:
        try:
            for ev in fetch(sport):
                r = find_arb(ev)
                if r is None: continue
                B, best = r
                margin = 1/B - 1
                stake = 200
```

```

        legs = " | ".join(
            f"{n} @ {o:.2f} ({b}): £{stake*(1/o)/B:.2f}"
            for n, (o, b) in best.items())
        notify(f"ARB {margin*100:.2f}% - {ev['home_team']} v {ev['away_team']}\n{legs}")
    except Exception as e:
        print("error:", e)
    time.sleep(60)

```

This is under 50 lines and will find real arbs. Everything in this document is structure and safety around this core.

20.2 Realistic roadmap

Week 1. Get the minimal script above working. Sign up for The Odds API free tier. Configure Telegram. Watch alerts for a week without placing any bets — you are calibrating your detector against reality.

Week 2. Add the dataclasses and modular structure from §4–9. Persist arbs to SQLite. Start manually placing small bets on high-margin, high-confidence arbs.

Week 3–4. Open Betfair, get an app key, install betfairlightweight, verify you can pull market data. Compare Betfair exchange odds to Odds API's Betfair prices to validate.

Month 2. Add Betfair placement for the hedge leg. Automate the exchange side; keep the soft-book side manual. Track void rate carefully.

Month 3+. Consider whether to graduate to value betting (§12 of the theory PDF). The infrastructure you have built transfers directly.

20.3 What to build next

- **Historical scanner-log analysis:** which books produce the most reliable arbs? Which sports? Rebuild the `book_allowed` list from data.
- **Line-tracking over time:** log every price change to identify slow books.
- **Pinnacle vig-free reference:** subtract vig from Pinnacle to get 'true' probability and start scoring soft-book prices for value.
- **Better name reconciliation:** track every failed match and build the alias table.
- **Web dashboard:** a small Flask/FastAPI app showing live arbs, deployed bankroll, and P&L by book and by sport.

Note. The whole system in this document is under 2,000 lines of Python. Do not add complexity you do not need. The bottleneck on your returns is not code quality — it is bookmaker limits, void rates, and time available. Spend accordingly.

End of Part II. Together with the theory volume, you now have everything needed to build, deploy, and operate a small-scale arbitrage betting system. The rest is practice, patience, and discipline about when to walk away.